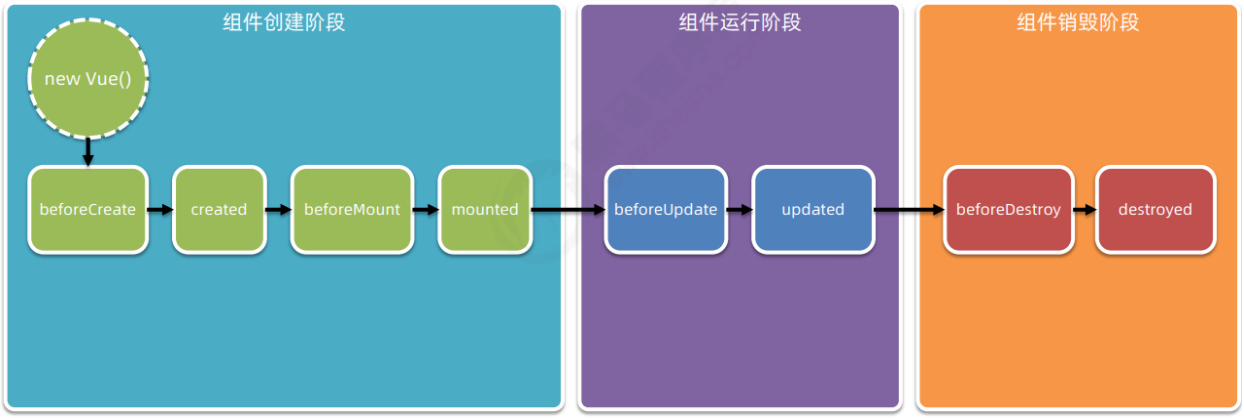
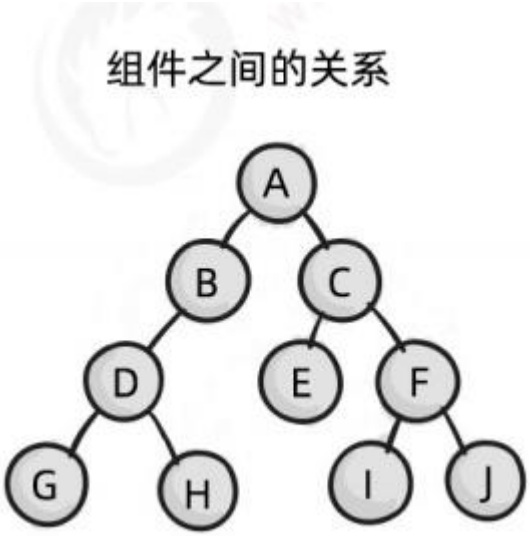
- **组件的生命周期**
  - 生命周期 & 生命周期函数
    - 生命周期（Life Cycle）是指一个组件从创建 -> 运行 -> 销毁的整个阶段，强调的是一个时间段。
    - 生命周期函数：是由 vue 框架提供的内置函数，会伴随着组件的生命周期，自动按次序执行。
    - 注意：生命周期强调的是时间段，生命周期函数强调的是时间点。
  - 组件生命周期函数的分类




组件生命周期的第 1 个阶段　　组件生命周期的第 2 个阶段　　组件生命周期的第 3 个阶段

  - 生命周期图示
    - 可以参考 vue 官方文档给出的“生命周期图示”，进一步理解组件生命周期执行的过程：
      - https://cn.vuejs.org/v2/guide/instance.html#生命周期图示

- **组件之间的数据共享**
  - 组件之间的关系
    - 在项目开发中，组件之间的最常见的关系分为如下两种：
      - 父子关系
      - 兄弟关系




  - 父子组件之间的数据共享
    - 父子组件之间的数据共享又分为：
      - 父 -> 子共享数据
      - 子 -> 父共享数据

- 父组件向子组件共享数据
  - 父组件向子组件共享数据需要使用自定义属性。示例代码如下：

```
// 父组件
<Son :msg="message" :user="userinfo"></Son>

data() {
  return {
    message: 'hello vue.js',
    userinfo: { name: 'zs', age: 20 }
  }
}
```

```
<template>
  <div>
    <h5>Son 组件</h5>
    <p>父组件传递过来的 msg 值是：{{ msg }}</p>
    <p>父组件传递过来的 user 值是：{{ user }}</p>
  </div>
</template>

props: ['msg', 'user']
```

- 子组件向父组件共享数据
  - 子组件向父组件共享数据使用自定义事件。示例代码如下：

子组件

```
export default {
  data() {
    return { count: 0 }
  },
  methods: {
    add() {
      this.count += 1
      // 修改数据时，通过 $emit() 触发自定义事件
      this.$emit('numchange', this.count)
    }
  }
}
```

父组件

```
<Son @numchange="getNewCount"></Son>

export default {
  data() {
    return { countFromSon: 0 }
  },
  methods: {
    getNewCount(val) {
      this.countFromSon = val
    }
  }
}
```

- 兄弟组件之间的数据共享
  - vue2.x 中，兄弟组件之间数据共享的方案是 EventBus。

```
import bus from './eventBus.js'

export default {
  data() {
    return {
      msg: 'hello vue.js'
    }
  },
  methods: {
    sendMsg() {
      bus.$emit('share', this.msg)
    }
  }
}
```

兄弟组件 A（数据发送方）

```
import Vue from 'vue'

// 向外共享 Vue 的实例对象
export default new Vue()
```

eventBus.js

```
import bus from './eventBus.js'

export default {
  data() {
    return {
      msgFromLeft: ''
    }
  },
  created() {
    bus.$on('share', val => {
      this.msgFromLeft = val
    })
  }
}
```

兄弟组件 C（数据接收方）

- EventBus 的使用步骤
  - 创建 eventBus.js 模块，并向外共享一个 Vue 的实例对象
  - 在数据发送方，调用 bus.$emit('事件名称', 要发送的数据) 方法触发自定义事件
  - 在数据接收方，调用 bus.$on('事件名称', 事件处理函数) 方法注册一个自定义事件

- **ref引用**
  - 什么是 ref 引用
    - ref 用来辅助开发者在不依赖于 jQuery 的情况下，获取 DOM 元素或组件的引用。
    - 每个 vue 的组件实例上，都包含一个 $refs 对象，里面存储着对应的 DOM 元素或组件的引用。默认情况下，组件的 $refs 指向一个空对象。

```
<template>
  <div>
    <h3>MyRef 组件</h3>
    <button @click="getRef">获取 $refs 引用</button>
  </div>
</template>

export default {
  methods: {
    getRef() { console.log(this) } // this 是当前组件的实例对象，this.$refs 默认指向空对象
  }
}
```

  - 使用 ref 引用 DOM 元素
    - 如果想要使用 ref 引用页面上的 DOM 元素，则可以按照如下的方式进行操作：

```
<!-- 使用 ref 属性，为对应的 DOM 添加引用名称 -->
<h3 ref="myh3">MyRef 组件</h3>
<button @click="getRef">获取 $refs 引用</button>

methods: {
  getRef() {
    // 通过 this.$refs.引用的名称 可以获取到 DOM 元素的引用
    console.log(this.$refs.myh3)
    // 操作 DOM 元素，把文本颜色改为红色
    this.$refs.myh3.style.color = 'red'
  },
}
```

  - 使用 ref 引用组件实例
    - 如果想要使用 ref 引用页面上的组件实例，则可以按照如下的方式进行操作：

```vue
<!-- 使用 ref 属性，为对应的“组件”添加引用名称 -->
<my-counter ref="counterRef"></my-counter>
<button @click="getRef">获取 $refs 引用</button>

methods: {
  getRef() {
    // 通过 this.$refs.引用的名称 可以引用组件的实例
    console.log(this.$refs.counterRef)
    // 引用到组件的实例之后，就可以调用组件上的 methods 方法
    this.$refs.counterRef.add()
  },
}
```

- 控制文本框和按钮的按需切换
  - 通过布尔值 inputVisible 来控制组件中的文本框与按钮的按需切换。示例代码如下：

```vue
<template>
  <input type="text" v-if="inputVisible">
  <button v-else @click="showInput">展示input输入框</button>
</template>
```

```vue
<script>
export default {
  data() {
    return {
      // 控制文本框和按钮的按需切换
      inputVisible: false,
    }
  },
  methods: {
    showInput() { // 切换布尔值，显示文本框
      this.inputVisible = true
    },
  },
}
</script>
```

- 让文本框自动获得焦点
  - 当文本框展示出来之后，如果希望它立即获得焦点，则可以为其添加 ref 引用，并调用原生 DOM 对象的.focus() 方法即可。示例代码如下：

```
<input type="text" v-if="inputVisible" ref="ipt">
<button v-else @click="showInput">展示input输入框</button>

methods: {
  showInput() {
    this.inputVisible = true
    // 获取文本框的 DOM 引用，并调用 .focus() 使其自动获得焦点
    this.$refs.ipt.focus()
  },
}
```

- this.$nextTick(cb) 方法
  - 组件的 $nextTick(cb) 方法，会把 cb 回调推迟到下一个 DOM 更新周期之后执行。通俗的理解是：等组件的DOM 更新完成之后，再执行 cb 回调函数。从而能保证 cb 回调函数可以操作到最新的 DOM 元素。

```
<input type="text" v-if="inputVisible" ref="ipt">
<button v-else @click="showInput">展示input输入框</button>

methods: {
  showInput() {
    this.inputVisible = true
    // 把对 input 文本框的操作，推迟到下次 DOM 更新之后。否则页面上根本不存在文本框元素
    this.$nextTick(() => {
      this.$refs.ipt.focus()
    })
  },
}
```